

TurnosRed — Informe técnico de la Actividad 3

Refactor de Especialidades y Profesionales hacia Clean Architecture: controllers async, validaciones con retorno anticipado y códigos de estado dinámicos · Integraciones Web

Repositorio: github.com/Luismcbo/turnos-red · **Stack:** Node.js + TypeScript + Express · **Autor:** Luis Valenzuela · **Fecha:** 24/09/2026

1. Resumen y punto de partida

El trabajo se hizo **sobre el proyecto existente turnos-red**. Al revisar el workspace no existían endpoints de **Especialidades** ni **Profesionales** (el proyecto tenía Turnos y Médicos), por lo que, siguiendo la consigna, se creó primero una **versión base “tradicional”** (rutas con handlers inline y arrays en memoria cargados desde `src/data/*.json`) y sobre ella se aplicó el refactor, como si fuera el resultado de la devolución de un Pull Request. Todo quedó verificado con la colección de Postman completa: **71 requests y 314 assertions, 0 fallas**.

Requisito de la consigna	Implementación	Dónde
1. Controllers por entidad; rutas limpias; controller general (hello + 404 final)	especialidades, profesionales y general.controller; las rutas solo mapean verbo + path; el 404 es un <i>app.use</i> final, no una ruta	src/controllers, src/routes
2. Async y estados dinámicos	Cada método es <i>export const x = async (req, res)</i> con <i>let status</i> ajustado según el flujo (200/201/204, 400/404, 500)	src/controllers
3. Validaciones previas y retorno anticipado	Validators de id, body y query; ante un fallo se fija <i>status</i> y se lanza <i>throw new Error(...)</i> ; todas las respuestas usan <i>return res.status(status).json()</i>	src/validators
4. try/catch robusto	El catch cubre errores propios y fallos inesperados (<i>status < 400</i> → 500) con el formato <i>{ status, message, code, details }</i>	utils/httpErrors.ts
5. Verificación en Postman	Colección con happy path y unhappy path forzados, más 404 general; corrida completa sin regresiones	postman/
6. README y árbol final	Documentación de endpoints, patrón de controllers y estructura de carpetas	README.md
7. Commits por etapa	5 commits separados (base, migración, validaciones, Postman, docs)	git log

2. Historial de commits (Actividad 3)

Commit	Mensaje
4c44129	feat: base tradicional de Especialidades y Profesionales (rutas inline)
f722685	refactor: migrar Especialidades y Profesionales a controllers async
0835912	feat: validaciones previas con throw y retorno anticipado en controllers
cd17715	test: colección de Postman para Especialidades y Profesionales (71 requests)
277104b	docs: README de la Actividad 3 (controllers async, endpoints y árbol final)

3. Arquitectura final: rutas limpias y controllers

Flujo: *routes* → *controllers* → *services* → *data (JSON en memoria)*. Las funciones de los servicios son async aunque los datos sigan en arrays, para que la futura integración con una base de datos no cambie el contrato con los controllers. Las rutas quedaron así:

```
import * as especialidadesController from '../controllers/especialidades.controller.js';

const router = Router();

router.get('/', especialidadesController.getAll);
router.get('/:id', especialidadesController.getById);
router.post('/', especialidadesController.create);
router.put('/:id', especialidadesController.update);
router.delete('/:id', especialidadesController.remove);

export default router;
```

general.controller.ts concentra `hello` (GET / → “Hello World”) y `notFound`, registrado con `app.use(generalController.notFound)` al final de la cadena (antes de los manejadores de errores), de modo que solo responde si ninguna ruta anterior lo hizo.

4. Controllers async, estado dinámico y return explícito

Ejemplo real (`especialidades.controller.ts` → `getById`). El patrón se repite en los 10 métodos de `Especialidades` y `Profesionales`:

```
export const getById = async (req: Request, res: Response) => {
  let status = 200;
  let details: ErrorDetail[] = [];
  try {
    const id = validateIdParam(req.params.id);
    if (id.data === undefined) {
      status = 400;
      details = id.problems;
      throw new Error('El id de la especialidad debe ser un entero positivo');
    }

    const especialidad = await especialidadesService.findById(id.data);
    if (!especialidad) {
      status = 404;
      throw new Error(`No existe una especialidad con id ${id.data}`);
    }

    return res.status(status).json(especialidad);
  } catch (error) {
    if (status < 400) status = 500;
    return res.status(status).json(toErrorBody(status, error, details));
  }
};
```

- **status** es una variable local que se ajusta al resultado real; si una validación falla se fija (400/404) *antes* de `throw new Error(...)`.
- El **catch** atiende ambos orígenes: si el status ya era 4xx es un error propio; si seguía siendo 2xx es un fallo inesperado y pasa a **500** (sin filtrar detalles internos, solo se registra en el log).
- Toda respuesta usa `return res.status(status).json(...)`: el *return* evita escribir la respuesta dos veces (“headers already sent”).

5. Validaciones previas y retorno anticipado

Antes de leer, crear, modificar, filtrar o eliminar, cada controller valida y corta el flujo con *throw*. Los `details` indican **qué campo falló y por qué**.

Validación	Status	Ejemplo
Body incompleto (campos obligatorios)	400	POST /especialidades {} → nombre obligatorio
Tipos inválidos y campos desconocidos	400	{ nombre: 123, descripcion: true, extra: "x" }
Id de ruta no numérico	400	GET /especialidades/abc
JSON mal formado	400	code INVALID_JSON
Filtros inválidos (query)	400	?especialidadId=abc&activo=quizas
Duplicados (nombre de especialidad, matrícula)	400	POST /especialidades {nombre:"Pediatria"}
Referencia inexistente	400	POST /profesionales {especialidadId: 999999}

Validación	Status	Ejemplo
Regla de negocio: especialidad con profesionales	400	DELETE /especialidades/2
Id inexistente / ruta inexistente	404	GET /especialidades/999999 · GET /ruta-inexistente
Caso feliz	200 / 201 / 204	DELETE devuelve 204 sin cuerpo

Evidencia (respuestas reales del servidor)

POST /especialidades { nombre: 123, descripcion: true, extra: "x" } → **400**, message: "Los datos de la especialidad no son válidos (3 errores)"

location	field	message
body	extra	Campo no permitido
body	nombre	Debe ser un texto (string)
body	descripcion	Debe ser un texto (string)

POST /profesionales { nombre: 1, apellido: "", matricula: null, especialidadId: "abc", email: "no-es-un-mail", activo: "si" } → **400** (6 detalles)

location	field	message
body	nombre	Debe ser un texto (string)
body	apellido	No puede estar vacío
body	matricula	Debe ser un texto (string)
body	email	Debe tener formato de correo electrónico
body	especialidadId	Debe ser un número entero positivo
body	activo	Debe ser booleano (true o false)

```
DELETE /especialidades/2 <- 400
{"status": 400, "message": "No se puede eliminar una especialidad con profesionales asociados", "code": "VALIDATION_ERROR", "details": [{"location": "params", "field": "id", "message": "Tiene 2 profesional(es) asociado(s)"]}]

GET /especialidades/999999 <- 404
{"status": 404, "message": "No existe una especialidad con id 999999", "code": "NOT_FOUND", "details": []}

GET /ruta-inexistente (404 general) <- 404
{"status": 404, "message": "Ruta no encontrada: GET /ruta-inexistente", "code": "NOT_FOUND", "details": []}
```

Caso feliz de eliminación: DELETE /especialidades/:id → **204**, cuerpo vacío (content-length: ausente).

6. Verificación en Postman (Newman)

Carpeta (orden de ejecución)	Requests	Assertions OK	Fallas
0. General (<i>nuevo</i>)	1	3/3	0
1. Médicos	11	51/51	0
2. Turnos	16	70/70	0
3. Limpieza y errores generales	4	15/15	0
4. Especialidades (<i>nuevo</i>)	15	67/67	0
5. Profesionales (<i>nuevo</i>)	16	74/74	0
6. Limpieza de Especialidades y Profesionales (<i>nuevo</i>)	8	34/34	0
Total	71	314/314	0

Colección postman/turnos-red.postman_collection.json: cada request verifica status code, *Content-Type* y esquema JSON; los ids (*especialidadId*, *profesionalId*) se encadenan con pm.environment.set. Corre en verde también sin environment y en re-ejecución.

Sin regresión: comparación contra la versión base

Se ejecutó la misma secuencia de 19 requests (happy paths) contra la versión base tradicional y contra el refactor: **15 de 19 respuestas son idénticas** (status y cuerpo). Las 4 diferencias son las intencionales:

Request	Base tradicional	Refactor
DELETE /profesionales/6	200 {"message": "Profesional eliminado"}	204 sin cuerpo
DELETE /especialidades/5	200 {"message": "Especialidad eliminada"}	204 sin cuerpo
GET /especialidades/5	404 {"error": "Especialidad no encontrada"}	404 {"status": 404, "message": "No existe una especialidad con id 5", "code": "...
GET /profesionales/999	404 {"error": "Profesional no encontrado"}	404 {"status": 404, "message": "No existe un profesional con id 999", "code": "...

Control negativo: la misma colección contra la versión base falla en **94 de 314 assertions** (no hay validaciones ni formato estándar), lo que confirma que los tests detectan el cambio de comportamiento.

7. Detalle de los requests nuevos (Newman)

Request	HTTP	OK
GET / - Hello World (200)	200	3/3
GET /especialidades - Listar especialidades (200)	200	3/3
GET /especialidades?nombre=pedia - Filtrar por nombre (200)	200	3/3
POST /especialidades - Crear especialidad (201)	201	4/4
GET /especialidades/:id - Obtener especialidad (200)	200	4/4
PUT /especialidades/:id - Actualizar especialidad (200)	200	4/4
POST /especialidades - Body incompleto (400)	400	5/5
POST /especialidades - Tipos de datos inválidos y campo descono...	400	5/5
POST /especialidades - Nombre duplicado (400)	400	5/5
POST /especialidades - JSON mal formado (400)	400	4/4
GET /especialidades?nombre=- - Filtro vacío (400)	400	5/5
GET /especialidades/abc - Id no numérico (400)	400	5/5
GET /especialidades/:id - Especialidad inexistente (404)	404	5/5
PUT /especialidades/:id - Especialidad inexistente (404)	404	5/5
PUT /especialidades/:id - Body vacío (400)	400	5/5
DELETE /especialidades/:id - Especialidad inexistente (404)	404	5/5
POST /profesionales - Crear profesional (201)	201	4/4
GET /profesionales - Listar profesionales (200)	200	4/4
GET /profesionales?especialidadId&activo&apellido - Filtrar (200)	200	3/3
GET /profesionales/:id - Obtener profesional (200)	200	4/4

Request	HTTP	OK
PUT /profesionales/:id - Actualizar profesional (200)	200	4/4
POST /profesionales - Body incompleto (400)	400	5/5
POST /profesionales - Tipos de datos inválidos (400)	400	5/5
POST /profesionales - Especialidad inexistente (400)	400	5/5
POST /profesionales - Matrícula duplicada (400)	400	5/5
PUT /profesionales/:id - Especialidad inexistente (400)	400	5/5
PUT /profesionales/:id - activo con tipo inválido (400)	400	5/5
GET /profesionales - Filtros inválidos (400)	400	5/5
GET /profesionales/abc - Id no numérico (400)	400	5/5
GET /profesionales/:id - Profesional inexistente (404)	404	5/5
PUT /profesionales/:id - Profesional inexistente (404)	404	5/5
DELETE /profesionales/:id - Profesional inexistente (404)	404	5/5
DELETE /especialidades/:id - Con profesionales asociados (400)	400	5/5
DELETE /profesionales/:id - Eliminar profesional (204)	204	2/2
GET /profesionales/:id - Profesional ya eliminado (404)	404	5/5
DELETE /profesionales/:id - Eliminar dos veces (404)	404	5/5
DELETE /especialidades/:id - Eliminar especialidad (204)	204	2/2
GET /especialidades/:id - Especialidad ya eliminada (404)	404	5/5
DELETE /especialidades/:id - Eliminar dos veces (404)	404	5/5
POST /ruta-que-no-existe - Ruta inexistente con otro verbo (404)	404	5/5

8. Uso de Inteligencia Artificial

Herramienta: **Claude Code** (Claude Sonnet 5) trabajando sobre el repositorio. La columna *Ajuste manual* queda para completar por quien entrega el trabajo.

Tarea	Herramienta	Prompt (resumen)	Respuesta generada	Ajuste manual
Refactor de Especialidades y Profesionales a controllers async con validaciones previas	Claude Code (Sonnet 5)	"Refactorizar hacia Clean Architecture: controllers por entidad, métodos async con variable status dinámica, validaciones con throw y return res.status(status).json(...) siempre, try/catch con { status, message, code, details }."	Al no existir la base se creó una versión tradicional y luego se migró en commits separados: controllers, services, controller general, validators y colección de Postman. Comparación contra la base (15/19 idénticas) y corrida Newman (314 assertions, 0 fallas; 94 fallan contra la base).	(a completar)

9. Decisiones de diseño y conclusión

- **Profesional vs. Médico:** se implementaron como recursos independientes (Médico se vincula a Turnos; Profesional a Especialidad).
- **Validación:** Turnos y Médicos siguen con Zod; Especialidades y Profesionales validan con *validators* propios desde el controller, como pide la consigna, y responden con el mismo formato de error.

- **GET /** devuelve el Hello World; por eso el cliente de Socket.IO pasó de / a */socket-client/*.
- **Referencia inválida** en el body (especialidadId inexistente) → 400; **id inexistente** en la ruta → 404. El estado vive en memoria.

Conclusión: cada endpoint responde desde su controller async, todas las respuestas usan *return* explícito, los códigos de estado se calculan según el resultado real del flujo y la colección completa (incluidos los casos de error forzados y el 404 general) corre en verde sin regresiones.